

Android 15 设置应用客制化指导手册

文档版本
发布日期

V1.0
2024-06-27

紫光展锐（上海）科技有限公司



版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍 Android 15 设置模块功能，内容涵盖展锐自研功能的客制化方法和常见问题的调试方法。

读者对象

本文档适用于所有需要在 Android 15 展锐平台，对设置模块进行客制化的研发人员。

缩略语

缩略语	英文全名	中文解释
CMCC	China Mobile Communications Corporation	中国移动
CTCC	China Telecom Communications Corporation	中国电信
FW	Frameworks	Android 架构层

符号约定

在本文中可能出现下列标志，它所代表的含义如下。

符号	说明
 说明	用于突出重要/关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2024-06-27	第一次正式发布

关键字

设置应用、智能控制、省电管理、定时开关机

目 录

1 概述.....	1
1.1 主界面.....	1
1.2 代码结构.....	2
2 自研功能客制化介绍.....	3
2.1 闪烁指示灯.....	3
2.1.1 功能介绍.....	3
2.1.2 修改文件清单.....	3
2.1.3 客制化配置.....	4
2.2 智能控制.....	4
2.2.1 功能介绍.....	4
2.2.2 修改文件清单.....	5
2.2.3 客制化配置.....	6
2.3 运行内存显示.....	7
2.3.1 功能介绍.....	7
2.3.2 修改文件清单.....	7
2.3.3 客制化配置.....	8
2.4 CP2 版本显示.....	8
2.4.1 功能介绍.....	8
2.4.2 修改文件清单.....	8
2.4.3 客制化配置.....	9
2.5 定时开关机.....	9
2.5.1 功能介绍.....	9
2.5.2 修改文件清单.....	9
2.5.3 客制化配置.....	10
2.6 恢复出厂设置低电量提醒.....	10
2.6.1 功能介绍.....	10
2.6.2 修改文件清单.....	11
2.6.3 客制化配置.....	11
2.7 省电管理.....	11
2.7.1 功能介绍.....	11
2.7.2 修改文件清单.....	15
2.7.3 客制化配置.....	15
2.8 色温和屏幕优化.....	15
2.8.1 功能介绍.....	15
2.8.2 修改文件清单.....	16
2.8.3 客制化配置.....	16

2.9 护眼模式	17
2.9.1 功能介绍	17
2.9.2 修改文件清单	17
2.9.3 客制化配置	17
2.10 Wi-Fi MAC 获取	18
2.10.1 功能介绍	18
2.10.2 修改文件清单	18
2.10.3 客制化配置	19
2.11 软硬件版本号	19
2.11.1 功能介绍	19
2.11.2 修改文件清单	20
2.11.3 客制化配置	21
2.12 本地系统升级	21
2.12.1 功能介绍	21
2.12.2 修改文件清单	22
2.12.3 客制化配置	22
3 常见问题说明	23
3.1 系统属性配置	23
3.2 时区	23
3.2.1 配置默认时区	23
3.2.2 默认时区不生效	24
3.2.3 时区添加或修改	25
3.3 语言	25
3.3.1 配置默认语言列表	25
3.3.2 配置语言支持列表	25
3.3.3 默认语言设置	27
3.4 搜索	28
3.4.1 删除一个界面的搜索	28
3.4.2 删除一个菜单的搜索	28
3.5 显示	29
3.5.1 Google 菜单的图标更换	29
3.5.2 开放源代码许可定制	30
3.6 版本号定制	32
3.6.1 软硬件版本号定制	32
3.6.2 内核版本号定制	32
3.7 数据库	33
3.7.1 默认字体大小修改	33
3.7.2 默认关闭系统所有动画	34
3.7.3 默认关闭快速打开相机	34

3.7.4 默认开启拿起手机即显示	35
3.7.5 默认关闭始终开启移动数据网络	35
3.7.6 配置第三方输入法为系统默认输入法	36

图目录

图 1-1 设置主界面	2
图 1-2 设置代码结构	2
图 2-1 闪烁指示灯开关界面	3
图 2-2 智能控制菜单及子界面	5
图 2-3 运行内存菜单	7
图 2-4 CP2 版本信息	8
图 2-5 定时开关机菜单及子界面	9
图 2-6 本地系统升级菜单及低电量提醒界面	10
图 2-12 色温和屏幕优化菜单与界面	16
图 2-13 护眼模式菜单与界面	17
图 2-14 Wi-Fi MAC 菜单	18
图 2-15 软件版本与硬件版本菜单	20
图 2-16 本地系统升级菜单与风险提示框	21
图 3-1 宏与系统属性生成位置	23
图 3-2 默认语言列表	25
图 3-3 语言支持列表	26
图 3-4 删除一个菜单的搜索	29
图 3-5 Google 菜单	29
图 3-7 内核版本号获取接口	33

表目录

表 2-1 智能控制依赖关系	4
表 2-2 智能控制开关列表	6

1 概述

设置应用是 Android 系统自带的一个比较重要的应用，给用户提供了 Android 系统中一些功能的设置入口。通过对一些功能进行设置，可以让用户个性化地使用自己的设备。

这些功能包含 Android 原生支持的功能和展锐自研的功能。

- 原生支持的功能
 - 网络与互联网
 - 设备连接
 - 应用与通知
 - 电池管理
 - 显示设置
 - 存储设置
 - 安全性设置
 - 账号设置以及语言
 - 输入法
 - 日期与时间
- 展锐自研的功能
 - 闪烁指示灯
 - 智能控制
 - 运行内存显示
 - CP2 版本显示
 - 定时开关机
 - 低电量提醒
 - 省电管理
 - 色温和屏幕优化
 - 护眼模式
 - Wi-Fi MAC 获取
 - 软硬件版本号
 - 本地系统升级
 - 动态导航栏

1.1 主界面

设置的主界面根据功能分类，划分了很多一级菜单项，如图 1-1 所示。

图1-1 设置主界面



1.2 代码结构

设置相关的代码主要包括原生功能和展锐自研功能的代码，这些代码的结构如图 1-2 所示。

图1-2 设置代码结构



2 自研功能客制化介绍

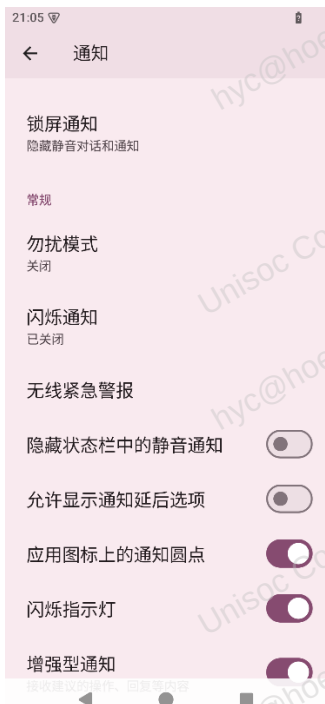
2.1 闪烁指示灯

2.1.1 功能介绍

Android 原生支持闪烁指示灯的功能，即设备收到通知时会闪烁通知。闪烁指示灯功能有一个功能开关 `config_intrusiveNotificationLed`，这个功能开关默认是关闭的。如果这个功能开关打开，设置应用中多处界面会显示闪烁指示灯的开关。但如果硬件上不支持指示灯功能，此时界面上显示的闪烁指示灯开关就没有任何实际功能。

故系统上设计了闪烁指示灯开关自适应功能，即在软件层面判断硬件是否支持闪烁指示灯功能，如果支持而且 `config_intrusiveNotificationLed` 开关是打开的状态，才显示闪烁指示灯开关；反之界面上不显示闪烁指示灯开关。目前设置应用中的闪烁指示灯开关界面如图 2-1 所示。

图2-1 闪烁指示灯开关界面



2.1.2 修改文件清单

- frameworks/base/core/res/res/values/config.xml
`<bool name="config_intrusiveNotificationLed">false</bool>`

AOSP 原生默认此值为 false

- vendor/sprd/platform/frameworks/base/core/res/res/values/config.xml
`<bool name="config_intrusiveNotificationLed">true</bool>`
 将闪烁指示灯功能开关 config_intrusiveNotificationLed 的默认值修改为 true。
- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/flashing/SettingsFlashingIndicatorControllerImpl.java
 方法 isRedLedNodeExist(), 判断当前设备的硬件是否支持指示灯功能。
- Settings/src/com/android/settings/notification/PulseNotificationPreferenceController.java
 控制图 2-1 闪烁指示灯开关显示的控制器。进入界面的路径为：“设置>通知”。在判断 config_intrusiveNotificationLed 开关值的基础上，再添加硬件是否支持指示灯功能的判断。
- Settings/src/com/android/settings/notification/app/LightsPreferenceController.java
 控制图 2-1 闪烁指示灯开关显示的控制器。进入界面的路径为：“设置>应用>所有应用>应用>通知>默认”。在判断 config_intrusiveNotificationLed 开关值的基础上，再添加硬件是否支持指示灯功能的判断。
- Settings/src/com/android/settings/notification/zen/ZenModeVisEffectPreferenceController.java
 控制图 2-1 不闪烁指示灯开关显示的控制器。进入界面的路径为：“设置>通知>勿扰模式>隐藏通知的显示方式>自定义”。在判断 config_intrusiveNotificationLed 开关值的基础上，再添加硬件是否支持指示灯功能的判断。
- Settings/src/com/android/settings/notification/zen/ZenRuleVisEffectPreferenceController.java
 控制图 2-1 不闪烁指示灯开关显示的控制器。进入界面的路径为：“设置>通知>勿扰模式>时间表>睡眠菜单设置图标>勿扰模式行为>为此时间表创建自定义设置>隐藏通知的显示方式>自定义”。在判断 config_intrusiveNotificationLed 开关值的基础上，再添加硬件是否支持指示灯功能的判断。

2.1.3 客制化配置

该功能沿用 Android 原生的配置开关 config_intrusiveNotificationLed，如果不需要该功能，可以将定义 isRedLedNodeExist() 方法的代码以及调用该方法的代码注释或者删除。

2.2 智能控制

2.2.1 功能介绍

该功能要求在设置应用首界面新增一个智能控制菜单，用于集合各种智能控制相关功能的设置开关。这些设置开关用于各个功能的打开与关闭，而相应功能的实现，依赖于各个业务模块的实现。它们的对应关系如表 2-1 所示，各个功能分布在不同的界面，具体的界面显示如图 2-2 所示。

表2-1 智能控制依赖关系

功能开关	描述	功能实现模块
黑屏唤醒	黑屏时，拿起手机并双击屏幕以点亮屏幕。	PhoneWindowManager
拿起手机即显示	拿起手机即可查看时间、通知和其他信息。	SystemUI

功能开关			描述	功能实现模块
智能体感	通话	体感拨号	拿起电话拨打屏幕上显示的联系人。	自研联系人
		体感接听	来电时拿起电话靠近耳朵自动接听。	Telecomm
		智能切换	语音通话免提时拿起手机靠近耳朵，自动切换为听筒。	自研 Dialer
		智能通话录音	通话中，首次摇一摇手机开始录音。	自研 Dialer
		来电响铃	来电响铃时拿起手机，铃声减弱并震动。	Telecomm
		来电静音	来电响铃时翻转手机转为静音。	Telecomm
口袋模式	口袋防误触		防止手机在口袋中引起误操作。	PhoneWindowManager
	铃声最大并震动		手机在口袋中来电，铃声自动最大并震动。	Telecomm
	自动关闭 Wi-Fi 搜索功能		手机在口袋中，自动关闭 Wi-Fi 的搜索功能来节省功耗。	Wi-Fi

图2-2 智能控制菜单及子界面



2.2.2 修改文件清单

智能控制作为一个功能开关集合，包含的内容很多，其总体实现包括 3 个部分。

- 定义各个功能开关在设置数据库中的字段名称

vendor/sprd/ex-interface/frameworks/base/core/java/android/provider/UnisocSettings.java

所有智能控制相关的各个功能的数据库字段在 UnisocSettings.java 中是定义在一起的，举例如下：

```
public static final String SMART_MOTION_ENABLED = "smart_motion_enabled"
```

```
public static final String SMART_BELL_SWITCH = "smart_bell_switch"
```

- 设置首界面添加菜单、添加智能控制功能开关相关
 - 添加智能控制各个 Sensor 相关功能的控制开关
 vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml
 因为与 Sensor 相关的功能很多，所以对应的功能开关也很多，具体如表 2-2 所示。

表2-2 智能控制开关列表

功能开关	对应功能菜单	默认值
config_support_smartControls	智能控制总开关	true
config_support_smartWake	黑屏唤醒开关	true
config_support_easyDial	体感拨号开关	true
config_support_easyAnswer	体感接听开关	true
config_support_handsfreeSwitch	智能切换开关	true
config_support_smartCallRecorder	智能通话录音开关	true
config_support_easyBell	来电响铃开关	true
config_support_muteIncomingCalls	来电静音开关	true
config_support_touchDisable	口袋防误触开关	true
config_support_smartBell	铃声最大并震动开关	true
config_support_powerSaving	关闭 Wi-Fi 搜索开关	true

- 添加智能控制菜单功能类 SmartControlsSettingsActivity 到设置应用原生代码中
 Settings/src/com/android/settings/Settings.java
 Settings/src/com/android/settings/core/gateway/SettingsGateway.java
- 添加智能控制菜单到设置首页
 Settings/res/xml/top_level_settings.xml
- 智能控制功能实现主体
 - vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/smartcontrols
 目录下的文件。
 - vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings
 还包括一些新增的资源文件等。

2.2.3 客制化配置

智能控制功能提供了统一的功能总开关和各项功能菜单的独立开关，便于进行定制。

- 关闭所有智能控制功能：配置总的开关 `config_support_smartControls` 的值为 `false`。
- 关闭其中某一项智能控制功能：配置该项子功能开关的值为 `false`。

2.3 运行内存显示

2.3.1 功能介绍

该功能要求在“设置>关于手机”界面，新增一个运行内存菜单，用于显示当前设备的运行内存，如图 2-3 所示。

图2-3 运行内存菜单



2.3.2 修改文件清单

- `vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml`
文件中新增了该功能的控制开关：`config_support_showRam`。
- `Settings/res/xml/my_device_info.xml`
添加运行内存菜单。
- `packages/apps/Settings/src/com/android/settings/deviceinfo/aboutphone/MyDeviceInfoFragment.java`
将控制菜单显示的控制器 `RamDisplayPreferenceController` 实例化，并添加到 `Controller` 列表。
- `vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/ramshow/RamDisplayPreferenceController.java`
运行内存功能菜单显示的控制器，根据控制开关 `config_support_showRam` 的值确定是否显示菜单。

2.3.3 客制化配置

- 功能控制开关 `config_support_showRam`，默认值为 `true`。
 - 打开运行内存显示功能：将值配置为 `true`。
 - 关闭运行内存显示功能：将值配置为 `false`。
- 定制 RAM 大小显示，该功能提供了一个系统属性 `ro.deviceinfo.ram`，当为该属性配置值时，优先显示该系统属性配置的 RAM 大小。例如，将该系统属性的值配置为 `1024000000`，RAM 大小会显示为 1GB。

2.4 CP2 版本显示

2.4.1 功能介绍

该功能要求在“设置>关于手机>Android 版本”界面，在基带版本信息的结尾添加 CP2 的版本信息，如图 2-4 所示。（具体以实际显示内容为准）

图2-4 CP2 版本信息



2.4.2 修改文件清单

- `vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml`
文件中新增了该功能的控制开关：`config_support_showCp2Info`。
- `packages/apps/Settings/src/com/android/settings/deviceinfo/firmwareversion/BasebandVersionPreferenceController.java`
控制基带版本信息内容显示的控制器，根据控制开关 `config_support_showCp2Info` 的值确定是显示基带版本信息（系统属性 `gsm.version.baseband` 的值），还是显示基带版本信息与 CP2 版本信息拼接的内容。
- `vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/version/SupportCPVersion.java`

用于获取并解析 CP2 版本信息，并拼接到基带版本信息的功能代码。

2.4.3 客制化配置

该功能提供的客制化配置是控制开关 `config_support_showCp2Info`，默认值为 `true`。

- 打开 CP2 版本显示功能：将值配置为 `true`。
- 关闭 CP2 版本显示功能：将值配置为 `false`。

2.5 定时开关机

2.5.1 功能介绍

该功能要求在设置应用首界面添加一个功能菜单定时开关机，实现定时开机和定时关机的功能。如图 2-5 所示，可以设置定时开机、关机的时间、重复周期等。

图2-5 定时开关机菜单及子界面



2.5.2 修改文件清单

定时开关机功能的实现主要包括 2 个部分。

- 设置首界面添加菜单，添加功能开关
 - `vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature_unisoc.xml`
新增功能控制开关：`config_support_scheduledPowerOnOff`。
 - `vendor/sprd/platform/packages/apps/Settings/Settings_Manifest.xml`

将定时开关机功能相关的文件添加到清单文件中。

- Settings/res/xml/top_level_settings.xml

设置首界面添加菜单。

- 定时开关机功能实现主体

- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/timerpower 目录下的文件，定时开关机功能的实现代码。
- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings 下新增的资源文件

2.5.3 客制化配置

该功能提供的客制化配置是控制开关 config_support_scheduledPowerOnOff，默认值为 true。

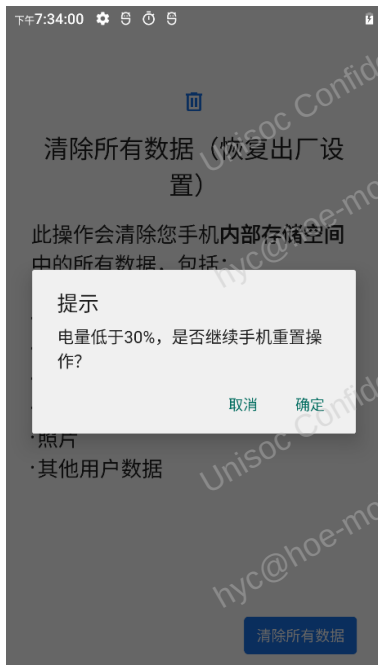
- 打开定时开关机功能：将值配置为 true。
- 关闭定时开关机功能：将值配置为 false。

2.6 恢复出厂设置低电量提醒

2.6.1 功能介绍

该功能要求在“设置>系统>重置选项>清除所有数据（恢复出厂设置）”界面进行恢复出厂设置操作时，如果电池电量低于 30%，需要提醒用户电量低，如图 2-6 所示。

图2-6 本地系统升级菜单及低电量提醒界面



2.6.2 修改文件清单

- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml
文件中新增了该功能的控制开关：config_support_lowBatteryFactoryResetPrompt。
- Settings/src/com/android/settings/MinClear.java
恢复出厂设置功能界面，在其中添加了显示提醒弹出框的代码，根据控制开关的值确定是否显示。

2.6.3 客制化配置

- 功能控制开关 config_support_lowBatteryFactoryResetPrompt，默认值为 true。
 - 打开恢复出厂设置低电量提醒功能：将值配置为 true。
 - 关闭恢复出厂设置低电量提醒功能：将值配置为 false。
- 弹出提醒的电池电量阈值 BATTERY_LEVEL_LIMIT，该值在 MasterClear.java 中定义，目前默认值为 30，可以通过修改 BATTERY_LEVEL_LIMIT 的值变更提醒弹出的时机。

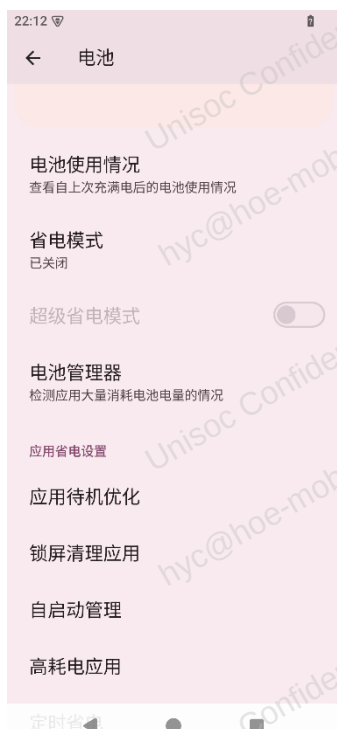
2.7 省电管理

2.7.1 功能介绍

该功能要求在“设置>电池”界面，增加展锐自研的一系列省电管理功能。

- 省电管理功能菜单，如图 2-7 所示。

图2-7 省电管理功能菜单



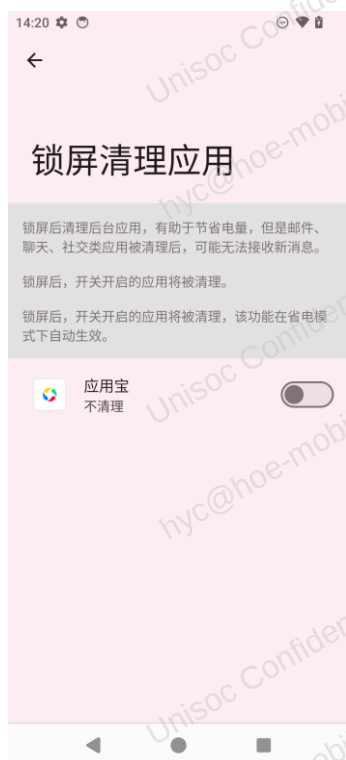
- 应用待机优化界面，如图 2-8 所示。

图2-8 应用待机优化界面



- 锁屏清理应用界面，如图 2-9 所示。

图2-9 锁屏清理应用界面



- 自启动管理界面，如图 2-10 所示。

图2-10 自启动管理界面



- 高耗电应用界面，如图 2-11 所示。

图2-11 高耗电应用界面



2.7.2 修改文件清单

- Settings/res/xml/power_usage_summary.xml
“设置>电池”界面的菜单定义文件，展锐自研的省电管理功能在该文件中定义了 5 个菜单
- Settings/src/com/android/settings/fuelgauge/PowerUsageSummary.java
“设置>电池”界面，在界面中添加判断，如果不支持展锐省电管理功能，直接移除 app_battery_saver_settings 以及其包含的 4 个菜单。
- SettingsLib/src/com/android/settingslib/applications/ApplicationsState.java
新增一个应用过滤器 UNISOC_FILTER_THIRD_PARTY。
- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml
各个子功能在 Settings 中的开关配置，具体如下：

```
<bool name="config_support_appStandbyOptimizer" translatable="false">true</bool>
<bool name="config_support_appAutoRunManagement" translatable="false">true</bool>
<bool name="config_support_lockScreenBatterySaver" translatable="false">true</bool>
<bool name="config_support_powerIntensiveApps" translatable="false">true</bool>
<bool name="config_support_superPowerSaving" translatable="false">true</bool>
```

- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/saving 目录下的文件
省电管理功能代码。
- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings 下对应的资源文件

2.7.3 客制化配置

该功能提供的客制化配置包括：

- 展锐自研的省电管理功能有一个功能总开关，总开关是由系统属性 persist.sys.pwctl.enable 控制，默认没有配置值。获取该系统属性时如果没有获取到值，默认返回 1，即该功能默认打开。如果不需要该功能，可以将 persist.sys.pwctl.enable 的值配置为 0。
- 设置应用中为各个子功能提供了单独的配置开关，如果需要支持某一子功能，就配置为 true，不需要支持就配置为 false。

2.8 色温和屏幕优化

2.8.1 功能介绍

该功能要求在“设置>显示”界面新增一个色温和屏幕优化菜单，提供三种颜色模式，用于调节屏幕的色域范围，如图 2-12 所示。

- 选择智能环境适应模式：可以手动通过色盘进行色温调节，也可以直接设置自然色，冷色，暖色三种色温。
- 选择其他颜色模式：色温调节色盘和三种色温选项都不可操作，其中三种色温选项菜单置灰。

图2-12 色温和屏幕优化菜单与界面



2.8.2 修改文件清单

- 设置>显示界面添加菜单，添加功能开关
 - vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml
新增功能控制开关：config_support_colorTemperatureAdjusting。
 - vendor/sprd/platform/packages/apps/Settings/Settings_Manifest.xml
将色温和屏幕优化功能相关的文件添加到清单文件中。
 - Settings/res/xml/display_settings.xml
显示界面添加色温和屏幕优化功能菜单。
- 色温和屏幕优化功能实现主体
 - vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/display 目录下的文件
颜色管理功能的实现代码。
 - vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings 下新增的资源文件

2.8.3 客制化配置

- Settings 显示控制开关 config_support_colorTemperatureAdjusting，默认值为 true。
 - 打开色温和屏幕优化功能：将值配置为 true。
 - 关闭色温和屏幕优化功能：将值配置为 false。

- 颜色管理功能开关 `ro.spr.d.displayenhance`，如果该系统属性配置为 `false`，即使设置应用中开关配置为 `true`，色温和屏幕优化功能也不支持。所以如果要支持色温和屏幕优化功能，两个开关需要同时配置为 `true`。

2.9 护眼模式

2.9.1 功能介绍

如图 2-13 所示，该功能要求在“设置>显示”界面，将 Android 原生的夜间模式菜单修改为护眼模式菜单，用于调整屏幕色温的浓度。该功能与颜色管理功能是互斥的，当启用该功能后，颜色管理界面的功能是禁用的。

图2-13 护眼模式菜单与界面



2.9.2 修改文件清单

- 修改菜单的标题，从夜间模式修改为护眼模式。
- 修改夜间模式的图标。
- 互斥的功能在 FW (Frameworks, Android 架构层) 中实现，不在设置中修改，此处不用详细说明。
- 将 Android 原生的开关 `config_nightDisplayAvailable` 配置为打开状态，原生默认为 `false`。

2.9.3 客制化配置

该功能提供的客制化配置是控制开关 `config_nightDisplayAvailable`，默认值为 `false`。

- 打开护眼模式功能：将值配置为 true。
- 关闭护眼模式功能：将值配置为 false。

2.10 Wi-Fi MAC 获取

2.10.1 功能介绍

Wi-Fi MAC 地址显示功能是 Android 原生的功能，只是原生功能在系统未开启过 Wi-Fi 的情况下，无法获取到 MAC 地址，只能显示未知。而 CTCC（China Telecom Communications Corporation，中国电信）入库要求在未开启过 Wi-Fi 的情况下必须能获取到 MAC 地址，因此提出了该功能的需求。该功能在设置部分的实现主要包括如下两点。

- 设计一个开关，只在 CTCC 版本上才生效，其他版本沿用 Android 原生功能。
- 提供在未开启 Wi-Fi 开关的情况下获取 Wi-Fi MAC 地址的功能。

Android 15 原生 Wi-Fi MAC 地址显示菜单名称为“设备 WLAN MAC 地址”，如图 2-14 所示。

图2-14 Wi-Fi MAC 菜单



2.10.2 修改文件清单

- vendor/sprd/platform/frameworks/base/packages/SettingsLib/res/values/config.xml
文件中新增了该功能的控制开关：config_enableGetWifiMacFromFile，默认为 false。
- vendor/sprd/platform/frameworks/base/SettingsLib/src/com/android/settingslib/deviceinfo/AbstractWifiMacAddressPreferenceController.java

添加了获取 Wi-Fi MAC 地址的逻辑。

- vendor/sprd/platform/frameworks/base/packages/SettingsLib/src/com/unisoc/settingslib/macaddress 路径下有文件
动态 overlay 用于在 CTCC 版本将控制开关 config_enableGetWifiMacFromFile 的值动态配置为 true。
- vendor/sprd/feature_configs/carriers/ctcc/config.mk
配置是否编译 CTCC 动态 overlay 的 apk。

2.10.3 客制化配置

该功能仅在 CTCC 版本生效，所以编译其他版本不包含此功能。

- CTCC 版本上去除该功能，需要在
vendor/sprd/platform/frameworks/base/packages/SettingsLib/res/values/config.xml 中，将控制开关配置为 false。
- 为了避免其他 overlay 的开关受影响，不能直接修改 config.mk 文件，将 overlay 的 apk 配置为不参与编译。

2.11 软硬件版本号

2.11.1 功能介绍

在设备上显示软件版本号和硬件版本号是 CTCC 以及 CMCC（China Mobile Communications Corporation，中国移动）入库的要求，该功能在 Settings 部分的实现主要包括如下几点。

- 在“设置>关于手机”界面新增一个软件版本菜单，用于显示软件版本号。
- 在“设置>关于手机”界面新增一个硬件版本菜单，用于显示硬件版本号。
- 设计一个开关，只在 CTCC 和 CMCC 版本上才生效，其他版本不显示软件版本菜单和硬件版本菜单。
- 在 CTCC 和 CMCC 版本上提供软件版本菜单和硬件版本菜单的搜索功能。

具体实现效果如图 2-15 所示。

图2-15 软件版本与硬件版本菜单



2.11.2 修改文件清单

- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/config_feattrue.xml
文件中新增了该功能的控制开关: config_enableSoftwareAndHardwareVersion, 默认为 false。
- Settings/res/xml/my_device_info.xml
添加软件版本号和硬件版本号菜单。
- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/version/HardwareRevisionPreferenceController.java
硬件版本菜单控制器。
- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/version/SoftwareRevisionPreferenceController.java
软件版本菜单控制器。
- vendor/sprd/feature_configs/carriers/ctcc/config.mk
配置是否编译 CTCC 动态 overlay 的 apk。
- vendor/sprd/carriers/ctcc/overlays/packages/apps/Settings/路径所有文件
动态 overlay 用于在 CTCC 版本将控制开关 config_enableSoftwareAndHardwareVersion 的值动态配置为 true。
- vendor/sprd/feature_configs/carriers/cmcc/config.mk
配置是否编译 CMCC 动态 overlay 的 apk。
- vendor/sprd/carriers/cmcc/overlays/packages/apps/Settings/路径所有文件

动态 overlay 用于在 CMCC 版本将控制开关 config_enableSoftwareAndHardwareVersion 的值动态配置为 true。

2.11.3 客制化配置

该功能仅在 CTCC 和 CMCC 版本生效，所以编译其他版本不包含此功能。

- CTCC 版本上去除该功能，需要在 vendor/sprd/carriers/ctcc/overlays/packages/apps/Settings/res/carrier_configs.xml 中将控制开关配置为 false。
- CMCC 版本上去除该功能，需要在 vendor/sprd/carriers/cmcc/overlays/packages/apps/Settings/res/carrier_configs.xml 中将控制开关配置为 false。
- 为了避免其他 overlay 的开关受影响，不能直接修改 config.mk 文件，将 overlay 的 apk 配置为不参与编译。

2.12 本地系统升级

2.12.1 功能介绍

该功能要求在设置>关于手机界面，新增一个本地系统升级菜单。当点击该菜单进行系统升级时，弹出对话框提示系统升级的风险，如图 2-16 所示。

图2-16 本地系统升级菜单与风险提示框



2.12.2 修改文件清单

- vendor/sprd/platform/packages/apps/Settings/res_unisoc_settings/values/config_feature.xml
文件中新增了该功能的控制开关：config_support_otaupdate。
- Settings/src/com/android/settings/deviceinfo/aboutphone/MyDeviceInfoFragment.java
将控制器 LocalSystemUpdatePreferenceController 实例化，并添加到 Controller 列表，通过功能控制开关 config_support_otaupdate 来控制该功能菜单是否可以被搜索到。
- vendor/sprd/platform/packages/apps/Settings/src_unisoc_settings/com/unisoc/settings/deviceinfo/LocalSystemUpdatePreferenceController.java
该功能菜单显示的控制器，根据控制开关 config_support_otaupdate 的值确定是否动态添加功能菜单。

2.12.3 客制化配置

该功能提供的客制化配置就是控制开关 config_support_otaupdate，默认值为 true。

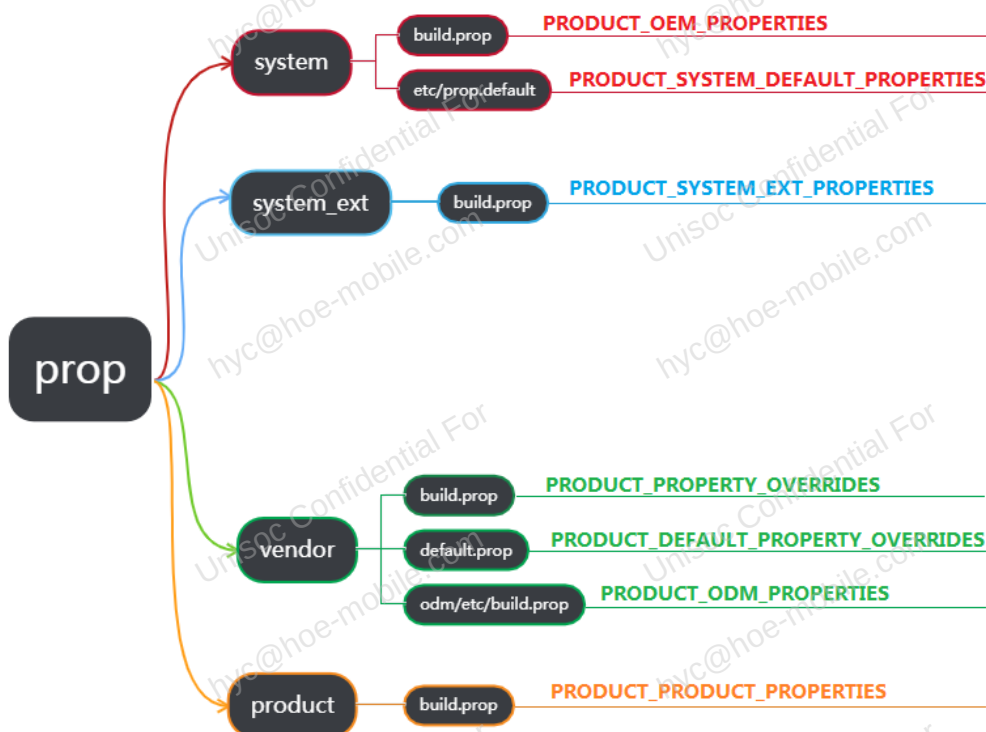
- 打开本地系统升级功能：将值配置为 true，
- 关闭本地系统升级功能：将值配置为 false。

3 常见问题说明

3.1 系统属性配置

不同的宏变量配置的系统属性，最终生成的路径是有区别的，具体如图 3-1 所示。每个分区的系统属性有相应的宏变量，通过设置这些宏变量就可以实现预置系统属性。

图3-1 宏与系统属性生成位置



3.2 时区

3.2.1 配置默认时区

设置默认时区，需要配置系统属性 `persist.sys.timezone` 的值。该系统属性需要配置在 `product` 分区，具体配置步骤如下。

步骤 1 在对应工程下面新建使用该系统属性模块目录，以 SC9863A 芯片的 s9863a3h10 工程为例，在 `device/sprd/sharkl3/s9863a3h10/module` 路径下创建模块的目录 `settings`。

步骤 2 在 settings 目录下创建 md.mk 文件。

步骤 3 在 md.mk 文件通过 PRODUCT_PRODUCT_PROPERTIES 宏将系统属性 persist.sys.timezone 配置到 product 分区的 build.prop 文件中，具体配置方法如下。

```
PRODUCT_PRODUCT_PROPERTIES += \
    persist.sys.timezone=Asia/Shanghai
```

----结束

3.2.2 默认时区不生效

配置的默认时区不生效，通常有如下几种情况：

- 自动更新时区默认开启，插卡开机，有网络连接，开机后自动更新了时区，导致默认时区不生效。
 - 情况说明：自动更新时区默认开启情况下，通过网络更新时区是正常的。如果不需要自动更新时区，可以默认关闭自动更新时区开关。
 - 修改方法：将自动更新时区开关 def_auto_time_zone 的值设置为 false。
 - 文件路径：SettingsProvider/res/values/defaults.xml。
- 自动更新时区默认开启，开机不插卡不连接网络，开机后自动更新了时区，导致默认时区不生效。
 - 情况说明：自动更新时区默认开启情况下，不插卡开机，不连接网络，设备也会驻留到紧急网络上，此时会根据紧急网络上报的国家码进行匹配并更新时区。
 - 修改方法：可以参考第 1 种情况的修改方法关闭自动更新时区开关。
- 设置的默认时区不在支持的时区列表中，显示了其他时区，导致默认时区不生效。
 - 情况说明：系统支持的默认时区列表在 tzlookup.xml 文件中，如果是 tzlookup.xml 中没有的时区 id，配置默认时区是无效的，因为系统不支持。
 - 修改方法：配置系统支持的时区。
 - 文件路径：system/timezone/output_data/android/tzlookup.xml。
- 系统带有 Google 开机向导。Google 开机向导中有时区设置界面，如果设置的默认时区不在其列表中，开机向导会显示其他时区，这时如果点击下一步，会将开机向导界面显示的时区设置到系统中，导致默认时区不生效。
 - 情况说明：Google 开机向导应用维护的时区列表是独立的，与系统支持的时区列表可能不一致。
 - 修改方法：Google 开机向导应用是闭源的，所以无法修改，可以配置 Google 开机向导应用支持的时区。
- SELinux 权限异常，导致默认时区不生效。
 - 情况说明：因为是权限问题，所以需要添加相应的权限。
 - 修改方法：在 device/sprd/mpool/sepolicy/vendor/vendor_init.te 文件中，添加权限方法如下。

```
allow vendor_init exported_system_prop:property_service { set }
```

说明

如果是因为 SELinux 权限问题不生效，kernel log 中会有如下信息打印，可以作为参考：

```
selinux: avc: denied { set } for property=persist.sys.timezone pid=1 uid=0 gid=0 scontext=u:r:vendor_init:s0
tcontext=u:object_r:exported_system_prop:s0 tclass=property_service permissive=0
```

```
.....init: Unable to set property 'persist.sys.timezone' to 'Europe/Amsterdam' in property file '/vendor/build.prop':  
SELinux permission check failed
```

3.2.3 时区添加或修改

Android 15 tzdata 已经 mainline，由 Google 推送更新，无法添加或者修改时区。

3.3 语言

3.3.1 配置默认语言列表

默认语言列表是指设置语言界面可以直接移动并进行设置的语言列表，不需要手动添加，如图 3-2 所示。

图3-2 默认语言列表

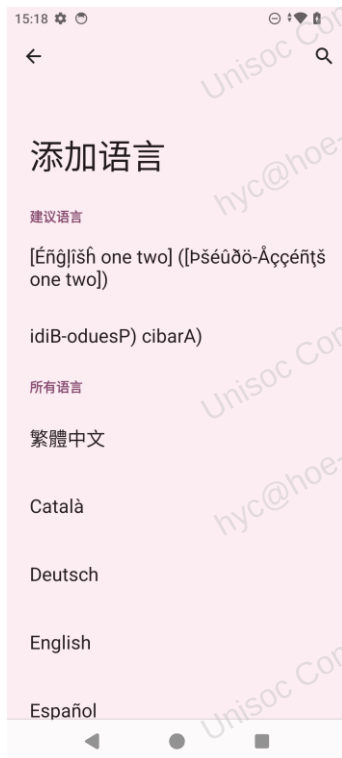


如果要配置默认语言列表，需要配置设置数据库中 Settings.System.SYSTEM_LOCALES 配置项的默认值，具体修改可以分为两种情况。

3.3.2 配置语言支持列表

语言支持列表是指添加语言界面可以添加的语言列表，如图 3-3 所示。

图3-3 语言支持列表



配置方法：

1. 需求确认

在以下文件中确认要添加的语言是否被原生支持。

frameworks/base/core/res/res/values/locale_config.xml

2. 添加语言，分两种情况：

– 如果没有配置 multi-lang

手机默认支持 Google 原生仓库，在语言系统中直接读取。

frameworks/base/core/res/res/values/locale_config.xml

build/target/product/languages_full.mk

– 如果已配置 multi-lang

若要添加一种语言（此语种需在原生或平台的支持范围内），需要修改以下文件。

vendor/sprd/feature_configs/multi-lang/overlay/frameworks/base/core/res/values/locale_config.xml

vendor/sprd/feature_configs/multi-lang/config.mk

例如添加 ar_IQ 语言，方法如下：

- i) 修改 vendor/sprd/feature_configs/multi-lang/overlay/frameworks/base/core/res/values/locale_config.xml 文件，将<!--<item>ar-IQ</item> -->修改为<item>ar-IQ</item>。
- ii) 修改 vendor/sprd/feature_configs/multi-lang/config.mk 文件，将 ar_IQ 添加到 FEATURES.PRODUCT_LOCALES。

3.3.3 默认语言设置

3.3.3.1 Native 版本

手机编译版本分为 Native 版本与 GMS 版本，Native 版本中不含 GMS 套件服务。

与 GMS 版本相比，Native 版本开机后没有开机向导，开机默认语言设置相对简单。插入 SIM 卡与不插 SIM 卡表现一致，均默认显示中文。

如需修改开机默认语言，可以在 vendor/sprd/feature_configs/carriers/ctcc/config.mk 文件中，修改以下配置，调换标黄部分语言包的顺序，将想要设置的语言包放在第一位，不管手机中是否有 SIM 卡，均显示设置的语言。

如下标黄部分，英语语言包处于第一位，开机后，不管手机中是否有 SIM 卡，开机后，均显示英语。

```
FEATURES.PRODUCT_PRODUCT_PROPERTIES += \
```

```
    ro.carrier=ctcc
```

```
    ro.product.locale= zh-Hans-CN \
```

```
    ro.product.country= CN
```

```
# add for cta feature
```

```
#FEATURES.PRODUCT_PRODUCT_PROPERTIES += \
```

```
#    ro.usb.support_cta=true
```

```
FEATURES.PRODUCT_LOCALES := en_US zh_Hans_CN
```

3.3.3.2 GMS 版本

开机默认语言设置

刷机后，进入开机向导，如果没有插入 SIM 卡，开机向导会自动切换默认语言。

经过本地测试，修改下列文件标黄部分，将想要设置的语言包放在 FEATURES.PRODUCT_LOCALES 属性第一位。

vendor/sprd/feature_configs/multi-lang/config.mk

```
FEATURES.PRODUCT_LOCALES := en_US zh_CN zh_HK zh_TW ar_EG fa_IR ru_RU fr_FR sw_TZ th_TH  
tr_TR es_ES es_US hi_IN in_ID vi_VN my_MM uk_UA pt_PT pt_BR as_ET ms_MY bn_BD tl_PH te_IN ta_IN  
ur_PK am_ET de_DE el_GR ml_IN mr_IN kn_IN hu_HU sq_AL fi_FI ca_ES eu_ES gl_ES km_KH lo_LA  
ne_NP si_LK or_IN pa_IN nl_NL it_IT ar_XB en_XA
```

```
FEATURES.PRODUCT_PACKAGE_OVERLAYS += \
```

```
    $(PRODUCT_REVISION_COMMON_CONFIG_PATH)/multi-lang/overlay
```

开机向导语言自适应

当手机中插入 SIM 卡时，开机向导的语言会由默认语言切换至对应 SIM 卡所属国家的语言。

Google 开机向导默认语言调用接口为 getSimLocale。

frameworks/base/telephony/java/android/telephony/TelephonyManager.java

```
@Nullable public Locale getSimLocale() {
    try {
        final ITelephony telephony = getITelephony();
        if (telephony != null) {
            String languageTag = telephony.getSimLocaleForSubscriber(getSubId());
            if (!TextUtils.isEmpty(languageTag)) {
                return Locale.forLanguageTag(languageTag);
            }
        }
        } catch (RemoteException ex) {
        }
    return null;
}
```

开机向导语言选择

在开机向导中，无论手机中是否插入 SIM 卡，都可以选择自己手机中语言，在开机向导语言列表中选择需要的语言包。

手机中可选择语言列表在下列文件中进行配置，在 FEATURES.PRODUCT_LOCALES 中添加或去掉相关的语言包。

配置文件：vendor/sprd/feature_configs/multi-lang/config.mk

3.4 搜索

3.4.1 删除一个界面的搜索

如果需要删除整个功能界面的搜索，直接将类名上面的注解删除即可。

以 DisplaySettings.java 为例，则删除以下行：

```
@SearchIndexable(forTarget = SearchIndexable.ALL & ~SearchIndexable.ARC)
```

3.4.2 删除一个菜单的搜索

SearchIndexProvider 中已实现 getNonIndexableKeys 方法，可以将对应菜单的 KEY 添加进去。

如果想删除这个菜单，只需要移除对应的 KEY，如图 3-4 所示。

图3-4 删除一个菜单的搜索

```
@Override
public List<String> getNonIndexableKeys(Context context) {
    final List<String> keys = super.getNonIndexableKeys(context);
    keys.add(KEY_SWIPE_DOWN);
    return keys;
}
```

3.5 显示

3.5.1 Google 菜单的图标更换

3.5.1.1 Google 菜单

Google 菜单如图 3-5 所示。

图3-5 Google 菜单



3.5.1.2 图标更换

设置首界面每个菜单都是一个 Preference，数据来源是一个与 Preference 绑定的 Tile 对象。在 Android 14 及以上版本，Tile 实现发生了变化，无法直接修改其中的 Icon 变量。因此如果要修改这个 Google 应用的图标，需要在 Preference 与 Tile 绑定的时候，判断 Tile 是否是 Google 菜单，如果是就给 Preference 设置期望的图标。

绑定 Tile 和 Preference 的实现在 DashboardFeatureProviderImpl.java 文件中，文件路径：Settings/src/com/android/settings/dashboard/DashboardFeatureProviderImpl.java。

修改方法是在 bindIcon 方法的最后添加更换图标代码。

```
ThreadUtils.postOnBackgroundThread() -> {
    String key = getDashboardKeyForTile(tile);
    if (!TextUtils.isEmpty(key)
        && key.endsWith("com.google.android.gms.app.settings.GoogleSettingsIALink")) {
        final Icon icon =
            Icon.createWithResource(mContext, R.drawable.ic_homepage_network);
        ThreadUtils.postOnMainThread() ->
            preference.setIcon(icon.loadDrawable(preference.getContext()))
    };
}
```

3.5.2 开放源代码许可定制

3.5.2.1 开放源代码许可说明

Android 15 显示开放源代码许可的菜单名称为“第三方许可”，“第三方许可”显示的内容来自 NOTICE.html 文件，该文件在手机的/data/user_de/0/com.android.settings/cache 路径下。NOTICE.html 文件由以下四个文件组合生成：

- /system/etc/NOTICE.xml.gz
- /vendor/etc/NOTICE.xml.gz
- /odm/etc/NOTICE.xml.gz
- /oem/etc/NOTICE.xml.gz

设置应用负责解析这些文件并组合生成最终的 NOTICE.html 文件。

3.5.2.2 开放源代码许可定制

NOTICE.html 最终是在设置应用的 LicenseHtmlGeneratorFromXml.java 中生成的。文件路径：frameworks/base/packages/SettingsLib/src/com/android/settingslib/license/LicenseHtmlGeneratorFromXml.java

如果需要在许可内容的开头和结尾添加特定的声明内容，方法如下：

1. 在 LicenseHtmlGeneratorFromXml 文件中添加两个常量，分别表示要求信息的开头和结尾字符串。

以下代码仅为示例。

```
private static final String HTML_CUSTOMER_HEADER_STRING =
    "<html><head>\n" +
    "<style type=\"text/css\">\n" +
    "body { padding: 0; font-family: sans-serif; }\n" +
    "same-license { background-color: #eeeeee;\n" +
```

```

"border-top: 20px solid white;\n" +
"padding: 10px; }\n" +
"label { font-weight: bold; }\n" +
"file-list { margin-left: 1em; color: blue; }\n" +
"</style>\n" +
"</head>" +

"<body topmargin=\"0\" leftmargin=\"0\" rightmargin=\"0\" bottommargin=\"0\">\n" +
"<div class=\"toc\">\n" +
"<table cellpadding=\"0\" cellspacing=\"0\" border=\"0\">\n" +
"<tr><td><!-- start-license --> \n" +
"<pre class=\"license-text\">\n" +
"客户自定义内容 \n" +
"</pre><!-- license-text -->\n" +
"</td></tr><!-- same-license -->\n" +
"</table>\n" +
"</div><!-- table of contents -->\n" +
"<div class=\"toc\">\n" +
"<ul>";

private static final String HTML_CUSTOMER_REAR_STRING =
    "<tr><td><!-- start-license -->\n" +
    "<pre class=\"license-text\">\n" +
    "客户自定义内容 \n" +
    "</pre><!-- license-text -->\n" +
    "</td></tr><!-- same-license -->\n" +
    "</table></body></html>";

```

- 在 LicenseHtmlGeneratorFromXml.java 文件的 **generateHtml** 方法中按红色字体进行修改。

```

@VisibleForTesting
static void generateHtml(Map<String, String> fileNameToContentIdMap,
    Map<String, String> contentIdToFileContentMap, PrintWriter writer,
    String noticeHeader) {
    List<String> fileNameList = new ArrayList();
    fileNameList.addAll(fileNameToContentIdMap.keySet());
    Collections.sort(fileNameList);
    writer.println(HTML_HEAD_STRING);
    //将HTML_HEAD_STRING替换为HTML_CUSTOMER_HEADER_STRING
    //中间省略部分代码
    writer.println(HTML_REAR_STRING);
}

```

```
//将 HTML_REAR_STRING 替换为HTML_CUSTOMER_REAR_STRING  
}
```

3.6 版本号定制

3.6.1 软硬件版本号定制

根据中国电信和中国移动入库需求，要求终端提供终端软件、硬件版本的查询。相关功能已导入到 Android 15 平台，但具体的软件版本号与硬件版本号需要自行配置。配置方法与 [3.2.1 配置默认时区](#) 基本相同，区别有如下两点。

- 需要找到支持中国电信或者中国版本对应的 board 进行配置。
- 软件版本号和硬件版本号对应的系统属性是 Settings 应用读取，而 Settings 应用在 Android 15 版本中最终在 system_ext 分区，所以系统属性也需要配置在 system_ext 分区。参考 [3.1 系统属性配置](#) 的介绍，软件版本号和硬件版本号系统属性使用宏 PRODUCT_SYSTEM_EXT_PROPERTIES 进行配置。属性值需要自行定义。具体示例如图 3-6 所示。

图3-6 软件和硬件版本号配置

```
PRODUCT_SYSTEM_EXT_PROPERTIES += \  
    ro.boot.hardware.revision=V1.00 \  
    ro.version.software=L1693.6.01.01.00
```

配置了软件、硬件版本号属性的设备，在“设置>关于手机”界面会显示配置的软件版本号与硬件版本号。

3.6.2 内核版本号定制

- 内核版本号显示路径：“设置>关于手机>Android 版本>内核版本”。
- 设置中获取内核版本号的接口：DeviceInfoUtils.getFormattedKernelVersion()，如 [图 3-7](#) 所示。
- 文件路径：SettingsLib/src/com/android/settingslib/DeviceInfoUtils.java。

如果希望设置中显示定制的内核版本号，可以修改该接口的返回值。

图3-7 内核版本号获取接口

```
public static String getFormattedKernelVersion(Context context) {
    return formatKernelVersion(context, Os.uname());
}

@VisibleForTesting
static String formatKernelVersion(Context context, StructUtsname uname) {
    if (uname == null) {
        return context.getString(R.string.status_unavailable);
    }
    // Example:
    // 4.9.29-g958411d
    // #1 SMP PREEMPT Wed Jun 7 00:06:03 CST 2017
    final String VERSION_REGEX =
        "(#\\d+)" + /* group 1: "#1" */
        "(?:.*?)" + /* ignore: optional SMP, PREEMPT, and any CONFIG_FLAGS */
        "((Sun|Mon|Tue|Wed|Thu|Fri|Sat).+)" + /* group 2: "Thu Jun 28 11:02:39 PDT 2012" */
    Matcher m = Pattern.compile(VERSION_REGEX).matcher(uname.version);
    if (!m.matches()) {
        Log.e(TAG, "Regex did not match on uname version " + uname.version);
        return context.getString(R.string.status_unavailable);
    }

    // Example output:
    // 4.9.29-g958411d
    // #1 Wed Jun 7 00:06:03 CST 2017
    return new StringBuilder().append(uname.release)
        .append("\n")
        .append(m.group(1))
        .append(" ")
        .append(m.group(2)).toString();
}
```

3.7 数据库

3.7.1 默认字体大小修改

1. 查看 Settings/res/values/arrays.xml，代码如下。

```
<string-array name="entries_font_size">
    <item msgid="6490061470416867723">Small</item>
    <item msgid="3579015730662088893">Default</item>
    <item msgid="1678068858001018666">Large</item>
    <item msgid="490158884605093126">Largest</item>
</string-array>

<string-array name="entryvalues_font_size" translatable="false">
    <item>0.85</item>
    <item>1.0</item>
    <item>1.15</item>
    <item>1.30</item>
</string-array>
```

从上述代码可知，Small 与 0.85、Default 与 1.0、Large 与 1.15、Largest 与 1.30 一一对应。

2. 字体大小配置项是 Settings.System.FONT_SCALE，在设置数据库的 System 表中。可以在 loadSettings 方法中修改默认值，如下所示。

在文件 SettingsProvider/src/com/android/providers/settings/DatabaseHelper.java 中添加如下代码。

// 1.30f对应最大字体，客户可自定义其他字体。

```
loadSettings(stmt, Settings.System.FONT_SCALE, 1.30f);
```

3.7.2 默认关闭系统所有动画

1. 在 WindowManagerService 中将默认系统动画属性值置为 0。

- 文件路径：
frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java。
- 修改方法：找到如下代码位置。

```
private float mWindowAnimationScaleSetting = 1.0f;
private float mTransitionAnimationScaleSetting = 1.0f;
private float mAnimatorDurationScaleSetting = 1.0f;
```

将以上 3 个系统动画属性值修改为“0.0f”。

```
private float mWindowAnimationScaleSetting = 0.0f;
private float mTransitionAnimationScaleSetting = 0.0f;
private float mAnimatorDurationScaleSetting = 0.0f;
```

2. 在设置数据库中，将动画的默认值设置为关闭。

- 文件路径：frameworks/base/packages/SettingsProvider/res/values/defaults.xml。
- 修改方法：找到如下代码位置。

```
<fraction name="def_window_animation_scale">100%</fraction>
<fraction name="def_window_transition_scale">100%</fraction>
```

将“def_window_animation_scale”和“def_window_transition_scale”的值修改为“0%”。

```
<fraction name="def_window_animation_scale">0%</fraction>
<fraction name="def_window_transition_scale">0%</fraction>
```

3.7.3 默认关闭快速打开相机

在 Android 15 上，快速打开相机菜单会有两种情况，可通过菜单的说明确认。

通过按音量上键两次快速打开相机

如果是双击音量上键开启相机，要默认关闭该功能，需要配置如下配置项的默认值。

- 配置项：Settings.Secure.CAMERA_DOUBLE_TAP_VOLUMEUP_GESTURE_DISABLED。
- 文件路径：SettingsProvider/src/com/android/providers/settings/DatabaseHelper.java。
- 修改方法：

该配置项在 Secure 表中，要在 loadSecureSettings 方法中添加如下代码，配置一下默认值：0 为默认开启，1 为默认关闭。

```
loadIntegerSetting(stmt,
Settings.Secure.CAMERA_DOUBLE_TAP_VOLUMEUP_GESTURE_DISABLED, 1);
```


通过按电源键两次快速打开相机

如果是双击电源键开启相机，要默认关闭该功能，需要配置如下配置项的默认值。

- 配置项: Settings.Secure.CAMERA_DOUBLE_TAP_POWER_GESTURE_DISABLED。
- 文件路径: SettingsProvider/src/com/android/providers/settings/DatabaseHelper.java。
- 修改方法:

该配置项在 Secure 表中，要在 loadSecureSettings 方法中添加如下代码，配置一下默认值：0 为默认开启，1 为默认关闭。

```
loadIntegerSetting(stmt,
Settings.Secure.CAMERA_DOUBLE_TAP_POWER_GESTURE_DISABLED, 1);
```

3.7.4 默认开启拿起手机即显示

拿起手机即显示即 Lift to check phone 功能，如果需要默认开启该功能，需要更改两个配置项的默认值，配置项: Settings.Secure.DOZE_PICK_UP_GESTURE 和 Settings.Secure.DOZE_ENABLED。

- 文件路径: frameworks/base/packages/SettingsProvider/res/values/defaults.xml。
- 修改方法: 找到如下代码位置。

```
<!-- Default for Settings.Secure.DOZE_PICK_UP_GESTURE -->
<bool name="def_doze_pick_up_gesture_enabled">false</bool>
<!-- Default for Settings.Secure.DOZE_ENABLED -->
<bool name="def_doze_enabled">false</bool>
```

将“def_doze_pick_up_gesture_enabled”和“def_doze_enabled”值修改为“true”。

```
<!-- Default for Settings.Secure.DOZE_PICK_UP_GESTURE -->
<bool name="def_doze_pick_up_gesture_enabled">true</bool>
<!-- Default for Settings.Secure.DOZE_ENABLED -->
<bool name="def_doze_enabled">true</bool>
```

3.7.5 默认关闭始终开启移动数据网络

始终开启移动数据网络是开发者选项界面中 Mobile data always active 功能，这个功能默认打开，当功能打开时，Wi-Fi 网络可以和移动网络共存。如果需要默认关闭这个功能，可以修改这个功能在数据库中对配置项的默认值，配置项如下。

Settings.Global.MOBILE_DATA_ALWAYS_ON。

- 文件路径: SettingsProvider/res/values/defaults.xml。
- 修改方法: 找到如下代码位置。

```
<!-- Default setting for Settings.Global.MOBILE_DATA_ALWAYS_ON -->
- <bool name="def_mobile_data_always_on">true</bool>
```

将“def_mobile_data_always_on”值修改为“false”:

```
<!-- Default setting for Settings.Global.MOBILE_DATA_ALWAYS_ON -->
<bool name="def_mobile_data_always_on">false</bool>
```


3.7.6 配置第三方输入法为系统默认输入法

配置第三方输入法为系统默认输入法，方法如下，以配置搜狗输入法为示例。

1. 将搜狗输入法预置到系统路径下，预置应用的方法此处不做详述。
2. 在预置了搜狗输入法的设备上设置手机输入法：设置>语言和输入法>屏幕键盘>管理屏幕键盘，打开搜狗输入法，并将搜狗输入法设为默认输入法。
3. 记录设置数据库 Secure 表中 DEFAULT_INPUT_METHOD 和 ENABLED_INPUT_METHODS 两个配置项对应的内容。可以通过如下命令进行获取。

```
adb shell settings get secure default_input_method //获取DEFAULT_INPUT_METHOD
```

```
adb shell settings get secure enabled_input_methods //获取ENABLED_INPUT_METHODS
```

4. 定义 DEFAULT_INPUT_METHOD 和 ENABLED_INPUT_METHODS 的默认值。

在 SettingsProvider/res/values/defaults.xml 中添加两个默认值配置项，用于定义默认值，默认值为 3 中记录的搜狗输入法的值，具体如图 3-8 所示。

图3-8 默认输入法配置

```
<!-- Default for Settings.Secure.DEFAULT_INPUT_METHOD -->
<string name="def_default_input_method" translatable="false">com.sohu.inputmethod.sogou/.SogouIME</string>

<!-- Default for Settings.Secure.ENABLED_INPUT_METHODS -->
<string name="def_enabled_input_methods" translatable="false">com.sohu.inputmethod.sogou/.SogouIME</string>
```

5. 将默认值加载到设置的数据库中。

因为默认输入法的两个配置项都在 Secure 表中，所以加载默认值在 DatabaseHelper.java 文件的 loadSecureSettings 方法中修改。

- 文件路径：SettingsProvider/src/com/android/providers/settings/DatabaseHelper.java。
- 修改方法：如图 3-9 所示。

图3-9 加载默认输入法到数据库

```
/* Set default input method @[ */
String defaultInput = mContext.getResources().getString(R.string.def_default_input_method);
if (!TextUtils.isEmpty(defaultInput)) {
    loadSetting(stmt, Settings.Secure.DEFAULT_INPUT_METHOD, defaultInput);
}
String enabledInputs = mContext.getResources().getString(R.string.def_enabled_input_methods);
if (!TextUtils.isEmpty(enabledInputs)) {
    loadSetting(stmt, Settings.Secure.ENABLED_INPUT_METHODS, enabledInputs);
}
/* @] */
```